

Proposal Report – Adaptive Scheduler (Contextual Bandit Module)

Abstract

Fixed-schedule study timers such as the Pomodoro method assume that all learners have the same cognitive endurance, yet research demonstrates that cognitive load fluctuates significantly within and across study sessions. Our subsystem introduces an Adaptive Scheduler driven by a Contextual Bandit algorithm. The bandit dynamically chooses work interval lengths (e.g., 20/30/45/60 minutes) and break durations (e.g., 3/5/8/12 minutes) based on the learner's real-time cognitive state. Unlike fixed schedules, the scheduler personalises cycles by balancing two rewards: task progress and post-break relief. Novel metrics are introduced, including Regret-per-Hour (RPH), Personalization Gain (PG), Adaptation Half-Life (AHL), Exploration Overhead Index (EOI), Break Utility Curve (BUC) with AUC-BUC, Counterfactual Targeting Uplift (CTU), and Stability-Productivity Frontier (SPF). These metrics capture efficiency, adaptation speed, and stability beyond traditional cumulative regret. By combining contextual bandit methods with productivity-focused metrics, this subsystem advances personalised learning technologies.

1. Introduction

1.1 Background and Literature Survey

Balancing focused work with timely breaks is central to productive study. However, current timers such as the Pomodoro method rigidly impose fixed cycles regardless of cognitive state, often interrupting deep focus or allowing fatigue to accumulate. The contextual bandit framework provides a powerful paradigm for adaptive decision-making under uncertainty, where an agent learns to balance exploration of new actions with exploitation of known effective ones [12]. Applications of contextual bandits in health [12], education [9], and recommendation systems [11] demonstrate their potential to personalise interventions. Nonetheless, existing productivity tools do not leverage contextual bandits or integrate novel evaluation metrics. Our Adaptive Scheduler subsystem extends prior research by implementing a contextual bandit that dynamically selects study-break intervals informed by cognitive load sensing, while evaluating outcomes with metrics tailored to learning contexts.

1.2 Research Gap

Existing study timers lack adaptation to individual cognitive states, relying on static intervals. Prior bandit research focuses on domains like recommendation systems and healthcare, but not on productivity scheduling. Furthermore, standard metrics such as cumulative regret inadequately capture long-term personalization, adaptation speed, or

stability. There is a gap in both methodology (bandit for study scheduling) and evaluation (novel productivity-focused metrics).

1.3 Research Problem

Students lack an adaptive, data-driven scheduling tool that respects cognitive variability and optimises both short-term relief and long-term productivity. The problem is twofold: (1) How can contextual bandits be used to dynamically schedule work and break intervals based on real-time signals, and (2) what new metrics can effectively evaluate such adaptive systems beyond standard regret-based evaluation?

2. Objectives

2.1 Main Objective

To design and implement an Adaptive Scheduler subsystem using contextual bandits to personalise work and break intervals, optimising productivity while reducing cognitive fatigue.

2.2 Specific Objectives

1. To design a contextual bandit framework for study scheduling that selects both work interval and break length.
2. To define and compute novel metrics (RPH, PG, AHL, EOI, BUC, CTU, SPF) tailored to study productivity.
3. To integrate the scheduler with cognitive load sensing data from keystroke dynamics and micro-EMA.
4. To evaluate the system in terms of adaptation efficiency, user acceptance, and long-term stability.

3. Methodology

3.1 Overall System Description

The Adaptive Scheduler subsystem receives cognitive load estimates from the sensing module. A contextual bandit then chooses the optimal work interval and break duration at each decision point. The bandit uses contextual features (typing speed, error rate, micro-EMA feedback, time-of-day) and updates its policy based on observed rewards. Rewards are computed as a weighted combination of (a) task progress during the interval, and (b) load relief after the break. This design ensures personalised cycles, unlike one-size-fits-all timers.

3.2 Implementation Steps

1. Context Definition: Capture contextual features including typing dynamics, self-reported micro-EMA scores, and chronotype information.
2. Action Space: Define work interval lengths (20/30/45/60 minutes) and break durations

(3/5/8/12 minutes).

3. Reward Function: Compute composite rewards combining post-break relief (measured by cognitive load estimator) and task progress metrics.
4. Bandit Algorithm: Implement and test algorithms (LinUCB, Thompson Sampling, Neural Bandits).
5. Metric Evaluation: Apply both standard (cumulative regret) and novel metrics (RPH, PG, AHL, EOI, BUC, CTU, SPF).
6. Integration: Provide API for scheduler decisions to interact with the overall study timer.
7. Evaluation: Conduct user studies to compare adaptive scheduling against fixed Pomodoro schedules.

4. Project Requirements

4.1 Functional Requirements

- Capture contextual features (typing speed, errors, EMA).
- Dynamically select work/break intervals via contextual bandit.
- Compute and log rewards after each cycle.
- Output personalised recommendations to user interface.

4.2 Non-Functional Requirements

- Privacy-preserving (no text content stored).
- Lightweight computation suitable for real-time use.
- Transparent reporting of why a break is suggested.
- Low user burden (minimal interruptions).

4.3 Expected Test Cases

1. System adapts intervals based on rising fatigue (detected by keystroke slowdown).
2. Scheduler recommends shorter breaks when load relief is achieved quickly.
3. Long-term test shows lower Regret-per-Hour compared to Pomodoro.
4. Break Utility Curve correctly shows diminishing returns for longer breaks.

5. Budget and Budget Justification

The subsystem requires minimal budget. Development uses open-source tools (Python, scikit-learn, bandit libraries). Cloud hosting is not required since processing can be done locally. Costs are limited to developer time, electricity, and internet. Estimated budget: negligible (< \$100).

6. Work Breakdown Structure (WBS)

The WBS follows the structure provided in the main group proposal (see shared section). Key stages include design, implementation, pilot testing, evaluation, and reporting. [Insert WBS Table Here]

7. References

- [8] Saito, Y., et al., Open Bandit Dataset and Pipeline (NeurIPS 2021 / arXiv 2020) — public logged bandit dataset (ZOZOTOWN).
- [9] Qin, C., Russo, D. (2022). On Adaptivity and Confounding in Contextual Bandit Experiments. ICML Workshop.
- [10] Lei, H., Lu, Y., Tewari, A., Murphy, S. (2017). An Actor-Critic Contextual Bandit Algorithm for Personalized Mobile Health Interventions. AAAI.
- [11] Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). A contextual-bandit approach to personalized news article recommendation. WWW.
- [12] Bouneffouf, D., & Rish, I. (2019). A Survey on Practical Applications of Multi-Armed and Contextual Bandits. arXiv.